



← Back to student guides

 DVC

Data versioning

3 levels

113 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

CODE

POINTER

CACHE

REMOTE



Quick Start

 Open PDF

A one-page starter for this tool. Scroll through it here, or open it full screen.



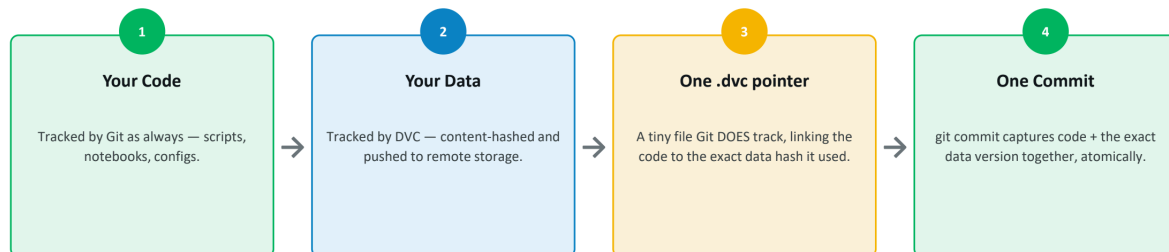


...and leaves behind a tiny pointer file
in Git that holds only a hash.

Git for code. DVC for data. Same commands, same habits, one workflow.

THE PROBLEM → THE FIX

How Git and DVC Divide the Work



01

Versioning Large Data with DVC



15

sections

39

examples

This is part two of three. It picks up exactly where Beginner ended and takes **every topic from there further**, then adds the machinery you have not met yet. Nothing is dropped and nothing is repeated for its own sake. Where you already know the basics, we go straight to the depth.

Where this picks up

Topic you already use	What this level adds
<code>.dvc</code> pointers	The hash algorithm, <code>files/md5/</code> layout, <code>.dir</code> listings, and why hashing is the bottleneck
The cache and link types	<code>reflink</code> vs <code>hardlink</code> vs <code>copy</code> measured, shared caches, and the corruption failure mode
<code>dvc add</code>	<code>--to-remote</code> , <code>--no-commit</code> , and when a directory should become shards instead
<code>dvc.yaml</code>	<code>foreach</code> , <code>matrix</code> , <code>vars</code> , templating, and multi-file pipelines
<code>outs</code>	<code>persist</code> , <code>cache: false</code> , <code>remote:</code> , <code>push: false</code> , and metrics-as-outputs
<code>params</code>	Parameter files other than <code>params.yaml</code> , <code>--set-param</code> sweeps, dotted paths
Metrics and plots	Plot templates, multi-revision comparison, DVCLive during training
Remotes	Multiple remotes, per-output remotes, <code>--jobs</code> , retries, and remote layout versions



<code>dvc import</code>	<code>import-url</code> , <code>--no-download</code> , <code>dvc update --rev</code> , and registry consumption
Experiments	Queues, <code>--temp</code> , custom runners, <code>dvc exp save</code> , and <code>exp show --sort-by</code>
Debugging	<code>dvc repro</code> internals, <code>--dry</code> , <code>--downstream</code> , <code>--force-downstream</code> , verbose transfer logs
new	CI/CD for pipelines · DVCLive · external outputs · monorepos · cloud-versioned data

Each section starts with the problem it solves, and ends with a **Try it** you can do on a real project in a few minutes.

Hashing, the cache layout, and why it matters

Beginner told you the cache is content-addressed. Performance questions start arriving at this level, and they all trace back to the mechanism.

DVC hashes file contents with **MD5** by default, chosen for speed rather than cryptographic strength, which is the honest answer when an interviewer asks. The hash becomes the object's address:

.dvc/cache layout

Copy

```
.dvc/cache/files/md5/
├── 8f/
│   └── 2c4b19e0a7d3f1c6b8a2e4d7091f3b    ← a file's contents
└── 3a/
    └── 7f9c2e8b1d4506....dir              ← a JSON listing for a directory
```

The first two hex characters become the directory, which keeps any single directory from holding millions of entries. A `.dir` object is a JSON array mapping relative paths to per-file



a .dir object

Copy

```
[
  {"md5": "a1b2c3...", "relpath": "cats/001.jpg"},
  {"md5": "d4e5f6...", "relpath": "cats/002.jpg"},
  {"md5": "789abc...", "relpath": "dogs/001.jpg"}
]
```

Three consequences follow, and each one shows up as a real problem:

Hashing is the bottleneck, not transfer. `dvc add` on a 50 GB dataset reads all 50 GB to compute hashes. On a slow disk that dominates everything. DVC caches file hashes by `(path, size, mtime, inode)` in `.dvc/tmp/`, so a `second` add of unchanged files is fast, which is why the first run after a `git clone` feels much slower than every run afterwards.

Small files are disproportionately expensive. A million 4 KB files means a million hash operations, a million cache entries, and a `.dir` listing of a million rows. The same bytes as 200 shard files is faster.

Deduplication is global to the cache. Identical content anywhere (two branches, two projects on a shared cache, two paths in one directory) occupies one object. That is why a shared cache across projects can cut disk usage sharply.

BASH

Copy

```
dvc config core.hash_algorithm md5           # the default
dvc config core.hash_algorithm sha256        # slower, cryptographically sound
dvc data status --granular                   # per-file view of what changed
du -sh .dvc/cache && find .dvc/cache -type f | wc -l
```

Changing the hash algorithm rewrites every address

Switching from md5 to sha256 changes every object's location, so your existing cache and remote become unusable for the new pointers. It is a migration, not a setting, so decide early, and if you need it for compliance reasons, do it before the dataset is large.



1. Generate two directories with the same total size: one with 20 files of 50 MB, one with 200,000 files of 5 KB.
2. Time `dvc add` on each. Compare.
3. Run `dvc add` a second time on the large-file one and time it again. The hash cache makes it near-instant.
4. Inspect the `.dir` object for the many-files case: `cat .dvc/cache/files/md5/<hash>.dir | head`.

Expected result – an order-of-magnitude difference for the same number of bytes, and a near-instant second run. Those two measurements are the entire argument for sharding small files, and they are far more convincing than being told.

Link types, measured

Beginner said `reflink,copy` is the safe default. On a large dataset the difference between the options is minutes and gigabytes.

Type	Extra disk	Speed	Safe to edit in place	Needs
<code>reflink</code>	None (copy-on-write)	Instant	Yes	APFS, Btrfs, XFS with reflink
<code>hardlink</code>	None	Instant	No: corrupts the cache	Same filesystem
<code>symlink</code>	None	Instant	No: corrupts the cache	Same filesystem
<code>copy</code>	Double	Proportional to size	Yes	Nothing



```
dvc config core.check_link_support true  
dvc doctor
```

```
# verify at runtime  
# what your filesystem supports
```

The `reflink,copy` ordering means "use copy-on-write if the filesystem can, otherwise make a real copy". That is the right default: no extra space where possible, and always safe.

The hardlink corruption failure, precisely

Under `hardlink`, your working file and the cache object are the same inode. Editing the file in place rewrites the cache object, so every commit that referenced that hash now silently points at different bytes, and `dvc checkout` on an old revision restores the `new` content while reporting success. It is a silent history corruption, which is why `cache.protected true` exists: it marks working files read-only so an in-place edit fails loudly instead.

The pattern for a machine where space is tight and speed matters:

BASH

Copy

```
dvc config --local cache.type hardlink,symlink  
dvc config --local cache.protected true  
# and then: never edit outputs in place, always regenerate through the pipeline
```

► TRY IT

1. Run `dvc doctor` and note which link types your filesystem supports.
2. Set `cache.type` to `copy`, add a large file, and measure `du -sh .dvc/cache`.
3. Switch to `reflink,hardlink`, run `dvc checkout --relink`, and measure again.
4. Set `cache.protected true`, then try to edit a tracked output in place and read the error.

Expected result – double the disk under `copy` and none under the link types, plus a hard permission error when you try to edit a protected file. That error is the safety net that makes hardlinks usable at all.



A pipeline with one stage per feature set, per model type, or per dataset shard quickly becomes hundreds of near-identical blocks. `foreach` and `matrix` collapse them.

dvc.yaml

Copy

```
stages:
  featurize:
    foreach:
      - tfidf
      - word2vec
      - embeddings
    do:
      cmd: python src/featurize.py --method ${item}
      deps:
        - src/featurize.py
        - data/prepared.csv
      outs:
        - data/features-${item}.pkl
```

That generates three stages named `featurize@tfidf` , `featurize@word2vec` , and `featurize@embeddings` . `foreach` also accepts a dictionary, which is how you vary several values at once:

YAML

Copy

```
stages:
  train:
    foreach:
      small:
        depth: 4
        estimators: 100
      large:
        depth: 16
        estimators: 500
    do:
      cmd: python src/train.py --depth ${item.depth} --estimators ${item.estimators}
      deps: [src/train.py, data/train.csv]
      outs: [models/model-${key}.pkl]
```



YAML

Copy

```
stages:
  benchmark:
    matrix:
      model: [rf, xgb, lgbm]
      split: [0.1, 0.2]
    do:
      cmd: python src/bench.py --model ${item.model} --split ${item.split}
      outs:
        - out/${item.model}-${item.split}.json
```

Six stages from six lines. `vars` lets you pull shared values in from a file or define them inline:

dvc.yaml

Copy

```
vars:
  - params.yaml                # everything in params.yaml is available
  - config/paths.yaml:data     # just the `data` key from another file
  - stage_defaults:
    timeout: 3600

stages:
  train:
    cmd: python src/train.py --timeout ${stage_defaults.timeout}
    deps:
      - ${data.train_path}
```

BASH

Copy

```
dvc dag                # confirm the generated stages
dvc repro featurize@tfidf  # target one generated stage
dvc repro 'featurize@*'   # or a glob
```

Generated stage names use @, and that matters for targeting

`featurize@tfidf` is the addressable name. Quote globs in the shell, and remember that `dvc repro featurize` with no suffix targets nothing, because the plain name does not



► TRY IT

1. Convert three near-identical stages into one `foreach` block and confirm `dvc dag` shows the same graph.
2. Run `dvc repro` and note the generated stage names in the output.
3. Add a second axis with `matrix` and count how many stages appear.
4. Move a shared path into `vars` from a separate YAML file and reference it with `${...}`.

Expected result – a much shorter `dvc.yaml` producing an identical DAG, and stage names with `@` suffixes. The moment you have more than three similar stages, this is the change that keeps the file readable.

Output modifiers

Beginner used plain `outs`. Every output takes modifiers, and choosing them correctly is most of what separates a pipeline that works from one that is pleasant to operate.

dvc.yaml

Copy

```
stages:
  train:
    cmd: python src/train.py
    outs:
      - models/model.pkl                # the default: cached, gitignored
      - models/summary.txt:             # small text → Git, diffable forever
        cache: false
      - logs/training.log:              # survives the next run; append-friendly
        persist: true
      - checkpoints/:                  # cached locally, never uploaded
        persist: true
        push: false
      - data/huge-intermediate.parquet: # cached to a specific remote
        remote: bigstore
```



<code>(none)</code>	Cached, gitignored, pushed	models, datasets, anything large
<code>cache: false</code>	Committed to Git instead	Metrics, plots, small summaries
<code>persist: true</code>	Not deleted before the stage reruns	Logs, checkpoints, incremental state
<code>push: false</code>	Cached locally, excluded from <code>dvc push</code>	Huge intermediates nobody else needs
<code>remote: NAME</code>	Stored on a named remote	Cost tiering, region pinning
<code>desc:</code>	Human description, shown in <code>dvc list</code>	Documenting a registry

Two of those deserve real attention.

`persist: true` changes the execution contract. Normally DVC removes a stage's outputs before running it, so the stage always starts clean. With `persist`, the previous output is left in place, which is what you need for checkpoint-resume training or an append-only log, and which also means the stage is no longer a function of its inputs. Use it deliberately.

`push: false` is the cost lever. A 200 GB intermediate that every machine regenerates in four minutes does not belong in object storage. Keeping it cached locally but out of the remote is often the single largest storage saving available.

BASH

Copy

```
dvc status          # persist outputs are reported differently
dvc repro --force    # persist outputs are still not deleted
dvc push --remote bigstore  # push only to a specific remote
```

`cache: false` means Git is now responsible

A `cache: false` output is committed to Git, so it must be small and text. DVC will let you mark a 4 GB binary `cache: false` and you will discover the consequence at `git push`. The rule of thumb: kilobytes and diffable, or it belongs in the cache.



1. Mark a small summary output `cache: false` , repro, and confirm it appears in `git status` rather than `.gitignore` .
2. Add a log output with `persist: true` , repro twice, and confirm the file was appended to rather than replaced.
3. Remove `persist` and repro again, and note that the file is now recreated from scratch.
4. Mark a large intermediate `push: false` , run `dvc push -v` , and confirm it was skipped.

Expected result – a small output living in Git, a log that survives across runs, and a push that skips the file you excluded. The `persist` before/after in steps two and three is the clearest demonstration of how it changes the stage contract.

External data and `import-url`

So far every input was a file in your repository. Real pipelines start from somewhere else: an S3 landing zone, an HTTP endpoint, a database export dropped on a share.

BASH

Copy

```
# Track a remote file as a dependency, with version tracking
dvc import-url s3://landing-bucket/exports/2024-05-01/events.parquet data/events.parquet

# HTTP works too
dvc import-url https://data.example.org/reference/taxonomy.csv data/taxonomy.csv

# Record the source without downloading the bytes now
dvc import-url --no-download s3://landing/huge.parquet data/huge.parquet

# Later, check whether upstream changed and pull it if so
dvc update data/events.parquet.dvc
dvc update --no-download data/huge.parquet.dvc
```

The `.dvc` file records both the source URL and the upstream object's own hash or etag, so `dvc update` can tell you whether the source changed:



```
deps:
- md5: 4f8a2b1c9e7d3506
  size: 1073741824
  path: s3://landing-bucket/exports/2024-05-01/events.parquet
outs:
- md5: 4f8a2b1c9e7d3506
  size: 1073741824
  path: events.parquet
```

You can also reference external data directly as a dependency inside a stage, which avoids a separate download step:

dvc.yaml

Copy

```
stages:
  prepare:
    cmd: python src/prepare.py
    deps:
      - src/prepare.py
      - s3://landing-bucket/exports/current/events.parquet # external dep
    outs:
      - data/prepared.parquet
```

Approach	Downloads	Tracks upstream version	Good for
<code>dvc import-url</code>	Yes	Yes, with <code>dvc update</code>	A source you want pinned and reproducible
<code>import-url --no-download</code>	No	Yes	Huge sources you process in place
External <code>deps</code> in a stage	No	Yes, by hashing the remote object	A moving landing zone read by the stage
<code>dvc get</code>	Yes	No	One-off downloads you do not depend on



overwrite silently invalidates your pipeline, which is sometimes exactly right and sometimes a reproducibility hole. If the result must be reproducible six months from now, import a **dated, immutable** path rather than a moving one.

► TRY IT

1. `dvc import-url` a small public HTTP file and read the resulting `.dvc` file's `deps` section.
2. Run `dvc update` on it and observe that nothing changes because upstream did not.
3. Add an external dependency to a stage and run `dvc status`, and note that DVC hashes the remote object to decide.
4. Try `--no-download` and confirm the pointer exists with no local file.

Expected result – a dependency that names an external source and knows its upstream version. Step three is the interesting one: DVC reaching out to hash a remote object is what makes external deps reproducible rather than convenient.

Multiple remotes

One remote is the common case. Several become necessary as soon as cost, geography, or access differ across your data.

BASH

Copy

```
dvc remote add -d origin s3://prod-bucket/dvc          # the default
dvc remote add archive s3://glacier-bucket/dvc         # cheap, cold
dvc remote add scratch /mnt/fast-nvme/dvc-scratch      # local, fast
dvc remote add readonly https://data.example.org/dvc   # HTTP, pull-only

dvc remote list
dvc push --remote archive
dvc pull --remote readonly
```



BASH

Copy

```
dvc remote modify origin region eu-west-1
dvc remote modify origin sse AES256 # server-side encryption
dvc remote modify origin acl bucket-owner-full-control
dvc remote modify origin jobs 16 # parallel transfers
dvc remote modify archive verify true # re-hash on download
dvc remote modify --local origin credentialpath ~/.aws/dvc-creds
dvc remote modify origin version_aware true # use bucket versioning
```

An output can name its own remote, which is how you tier storage without moving anything by hand:

dvc.yaml

Copy

```
outs:
  - models/model.pkl # → default remote
  - data/raw-archive.parquet:
    remote: archive # → cold storage
```

Setting	Why it matters
<code>jobs</code>	Transfer parallelism. The single biggest <code>dvc pull</code> speed lever
<code>sse</code> / <code>sse_kms_key_id</code>	Encryption at rest, usually a compliance requirement
<code>verify</code>	Re-hash after download; catches corruption at the cost of time
<code>version_aware</code>	Store objects at their natural paths and use the bucket's own versioning
<code>credentialpath</code> / <code>profile</code>	Per-machine credentials, set with <code>--local</code>

Tune `jobs` before blaming the network

DVC's default parallelism is conservative. On a fat link with many small objects, `dvc remote modify origin jobs 16` or `dvc pull -j 16` often multiplies throughput. Measure



► TRY IT

1. Define a second remote pointing at a different local directory and push to it explicitly with `--remote`.
2. Mark one output with `remote:` naming that second remote, repro, and push. Confirm where each object landed.
3. Time `dvc pull -j 1` against `dvc pull -j 16` on a directory with many small files.
4. Run `dvc remote list -v` and read the resolved configuration.

Expected result – objects split across two remotes according to a per-output rule, and a measurable difference from transfer parallelism. Measure `jobs` yourself once and you will know your own numbers instead of guessing.

Experiments at scale

Beginner ran a few experiments in sequence. At this level you want dozens, running in parallel, without three of them fighting over the same workspace.

BASH

Copy

```
# Queue a sweep from a range
dvc exp run --queue -S train.n_estimators=100,200,400,800
dvc exp run --queue -S 'train.max_depth=range(4, 20, 4)'

# Run the queue with N workers
dvc exp run --run-all --jobs 4

# Watch and manage the queue
dvc queue status
dvc queue logs exp-a1b2c
dvc queue stop
dvc queue remove --all
```



implicitly, and what makes parallel sweeps safe.

BASH

Copy

```
dvc exp run --temp # run in a temp dir, leave your workspace alone
dvc exp run --temp --set-param train.lr=0.01
```

Comparing and filtering becomes the main activity once you have thirty results:

BASH

Copy

```
dvc exp show --only-changed # hide constant columns
dvc exp show --sort-by accuracy --sort-order desc
dvc exp show --num 5 # last 5 commits' experiments
dvc exp show --drop 'train_seconds|.*_std'
dvc exp show --csv > results.csv # or --json, --md
dvc exp diff exp-a1b2c exp-d4e5f # two experiments, side by side
```

Promotion has three shapes, each appropriate to a different situation:

Command	Result	Use when
<code>dvc exp apply NAME</code>	Brings it into your workspace, uncommitted	You want to inspect then commit
<code>dvc exp branch NAME BRANCH</code>	Creates a branch at that experiment	It deserves a pull request
<code>dvc exp save -n NAME</code>	Snapshots the current workspace as an experiment	You changed things by hand and want to keep it
<code>dvc exp push origin NAME</code>	Shares it via Git refs	A colleague should see your result

BASH

Copy



`dvc exp pull origin exp-a1b2c`

`# fetch theirs`

Experiments are Git refs, and they accumulate

Each experiment is a hidden commit under `refs/exps/` plus cached outputs. Thirty sweeps of a 2 GB model is 60 GB of cache. `dvc exp remove` drops the refs and `dvc gc --workspace` (or better, `--all-commits`) reclaims the space, but neither happens automatically, and "my disk filled up during a sweep" is the most common mid-level surprise.

► TRY IT

1. Queue a sweep with `-S param=a,b,c,d` and run it with `--jobs 2`. Watch `dvc queue status` in another terminal.
2. Run `dvc exp show --only-changed --sort-by` your metric, and identify the winner.
3. Measure your cache before and after with `du -sh .dvc/cache`.
4. Promote the winner with `dvc exp branch`, then clean up with `dvc exp remove --all` and a dry-run `dvc gc`.

Expected result – a parallel sweep that does not corrupt itself, a sorted comparison table, and a visible jump in cache size. Step three is the one that teaches you to budget disk for a sweep before starting it.

DVCLive: logging from inside training

`metrics.json` written at the end of a run tells you the final score. **DVCLive** logs during the run, which gives you per-epoch curves and lets you compare training dynamics rather than just endpoints.

BASH

Copy



src/train.py

Copy

```
from dvclive import Live

with Live() as live:
    live.log_params({"lr": lr, "batch_size": bs, "epochs": epochs})

    for epoch in range(epochs):
        train_loss = train_one_epoch(model, loader)
        val_loss, val_acc = validate(model, val_loader)

        live.log_metric("train/loss", train_loss)
        live.log_metric("val/loss", val_loss)
        live.log_metric("val/accuracy", val_acc)
        live.next_step()                # advances the step counter

    live.log_artifact("models/model.pkl", type="model", name="classifier")
```

DVCLive writes into `dvclive/` in a layout DVC already understands, so the pipeline wiring is short:

dvc.yaml

Copy

```
stages:
  train:
    cmd: python src/train.py
    deps: [src/train.py, data/train.csv]
    params: [train.lr, train.batch_size, train.epochs]
    outs:
      - models/model.pkl
    metrics:
      - dvclive/metrics.json:
        cache: false
    plots:
      - dvclive/plots/metrics:
        x: step
        cache: false
```



```
dvc plots show          # per-epoch curves
dvc plots diff HEAD~1   # this run's curves against the last
dvc exp show            # final values appear as columns automatically
```

There are framework integrations that remove the manual loop entirely: callbacks for PyTorch Lightning, Keras, Hugging Face, XGBoost, LightGBM, and more:

PYTHON

Copy

```
from dvclive.lightning import DVCLiveLogger
trainer = Trainer(logger=DVCLiveLogger(log_model=True))
```

DVCLive is the seam between "a number" and "a curve"

A final accuracy tells you which run won. A loss curve tells you **why**: overfitting after epoch 12, a learning rate that was too high, a validation set that is too small to be stable. The cost is about five lines, and it changes what your experiment table is useful for.

► TRY IT

1. Add DVCLive to a training script with three `log_metric` calls in the epoch loop.
2. Wire `dvclive/metrics.json` and `dvclive/plots/metrics` into `dvc.yaml` with `cache: false`.
3. Run `dvc repro`, then `dvc plots show` and open the HTML.
4. Change the learning rate, repro, then `dvc plots diff HEAD~1` and compare the two curves on one chart.

Expected result – two training runs overlaid on a single loss chart. That comparison is what makes a hyperparameter change explainable rather than measurable, and it is the artifact to put in a pull request.

Plot templates and comparison



dvc.yaml

Copy

```
plots:
  - plots/confusion.csv:
      template: confusion
      x: actual
      y: predicted
      cache: false
  - plots/roc.json:
      template: linear
      x: fpr
      y: tpr
      title: ROC curve
      x_label: False positive rate
      y_label: True positive rate
      cache: false
  - plots/importance.csv:
      template: bar_horizontal
      x: importance
      y: feature
      cache: false
```

Template	Shows
<code>linear</code>	The default; a line chart
<code>confusion</code>	A confusion matrix heatmap
<code>confusion_normalized</code>	The same, as proportions
<code>scatter</code>	Point cloud
<code>smooth</code>	Line with a smoothing window
<code>bar_horizontal</code>	Ranked bars, ideal for feature importance

Top-level plot definitions let you combine several files into one chart, which is how you compare across stages or models:



```
plots:
  - accuracy-comparison:
      x: epoch
      y:
        logs/rf.csv: accuracy
        logs/xgb.csv: accuracy
      title: RF versus XGBoost
```

BASH

Copy

```
dvc plots show # write and open an HTML report
dvc plots diff HEAD~3 HEAD --targets plots/roc.json
dvc plots diff --open # open in a browser directly
dvc plots templates # list available templates
dvc plots templates confusion --out .dvc/plots # customise one
```

Plots are for pull requests, not for you

You already know what your model does. The plot exists so a reviewer can see the difference in ten seconds without running anything. Optimise for that: label the axes, give it a title, and make `dvc plots diff` against the base branch part of your CI output.

► TRY IT

1. Write a confusion matrix CSV from your evaluate stage and declare it with `template: confusion`.
2. Run `dvc plots show --open` and check the rendering.
3. Add a top-level plot definition combining two files onto one chart.
4. Run `dvc plots templates` and try a second template on the same data.

Expected result – a rendered confusion matrix and a combined chart from two sources, with no plotting code written. The templates are the reason to use DVC plots rather than saving PNGs: the data stays diffable and the rendering is declarative.



A pipeline that only reproduces on your laptop is not reproducible. Putting `dvc repro` in CI turns reproducibility from a claim into a check.

.github/workflows/pipeline.yml

Copy

```
name: Pipeline
on:
  pull_request:
  push:
    branches: [main]

jobs:
  repro:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0 # DVC needs history for metrics diff

      - uses: actions/setup-python@v5
        with:
          python-version: '3.11'
          cache: pip

      - run: pip install -r requirements.txt

      # Credentials via OIDC – no stored keys
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: ${ vars.DVC_ROLE_ARN }
          aws-region: ${ vars.AWS_REGION }

      # Cache DVC's own cache between runs, keyed on the lock file
      - uses: actions/cache@v4
        with:
          path: .dvc/cache
          key: dvc-${ hashFiles('dvc.lock') }
          restore-keys: dvc-

      - run: dvc pull --allow-missing
      - run: dvc repro
```



```
dvc status --json | tee status.json
test "$(cat status.json)" = "{}" || {
  echo "::error::dvc.lock is stale – run 'dvc repro' and commit"; exit 1; }

- name: Report metrics on the pull request
  if: github.event_name == 'pull_request'
  run: |
    {
      echo "## Metrics"
      dvc metrics diff origin/main --md
      echo "## Parameters"
      dvc params diff origin/main --md
    } >> "$GITHUB_STEP_SUMMARY"

- run: dvc push
  if: github.ref == 'refs/heads/main'
```

Five details in there carry the weight:

fetch-depth: 0 . `dvc metrics diff origin/main` needs the other revision present. A shallow clone silently produces an empty diff.

Caching `.dvc/cache` **keyed on** `dvc.lock` . Without it, every CI run downloads every input. With it, an unchanged pipeline pulls almost nothing.

`dvc pull --allow-missing` . Pulls what exists and does not fail on outputs that have not been produced yet, which is the normal state on a branch.

The `dvc status` **gate**. This is the check that matters: it fails the pull request when someone edited a script or a parameter without rerunning the pipeline, so `dvc.lock` cannot drift from the code.

`dvc push` **only on** `main` . Feature branches produce experiments, not canonical artifacts. Pushing from every branch fills your remote with dead ends.

CI concern	Answer
Credentials	OIDC or a workload identity: never a stored key



Cold-start cost	Cache <code>.dvc/cache</code> , keyed on <code>dvc.lock</code>
"The lock is stale"	<code>dvc status</code> as a required check
Reviewer visibility	<code>dvc metrics diff --md</code> into the job summary
Who may push data	Only the protected branch
Long-running training	A self-hosted or GPU runner: Senior covers this

A fork pull request must not receive push credentials

The same trust model as any CI system: a pull request from a fork runs contributor code. If that job holds a write credential for your data remote, a contributor can overwrite your datasets. Give fork builds a read-only remote, or split the workflow so the privileged half never runs fork code.

► TRY IT

1. Add the workflow above to a repository, adapted to your remote, and open a pull request that changes a parameter.
2. Confirm the metrics diff appears in the job summary.
3. Now change a script

Expected result – without running `dvc repro`, push, and confirm the `dvc status` gate fails.

4. Compare run times with and without the `actions/cache` step.

Expected result – a pull request that shows how the numbers moved, and a red check when the lock file is stale. That failing gate in step three is the single most valuable thing this workflow does. It makes "reproducible" enforceable rather than aspirational.



One repository with several models, or one dataset shared by several repositories, both need a layout decision.

Subdirectory pipelines. DVC supports a `dvc.yaml` per subdirectory, each with its own stages, sharing one `.dvc/cache` and one remote:

monorepo layout

Copy

```
.
├── .dvc/                    # one cache, one remote, one config
├── projects/
│   ├── churn/
│   │   ├── dvc.yaml
│   │   ├── dvc.lock
│   │   └── params.yaml
│   └── fraud/
│       ├── dvc.yaml
│       ├── dvc.lock
│       └── params.yaml
└── shared/
    └── data/features.parquet.dvc
```

BASH

Copy

```
dvc repro projects/churn/dvc.yaml
dvc repro --all-pipelines          # every dvc.yaml in the repo
dvc dag projects/fraud/dvc.yaml
```

A stage in one project can depend on an output from another by relative path, which makes shared feature engineering a first-class dependency rather than a copy.

Separate repositories with `import` . When teams need independence, keep the data in its own repository and consume it:

BASH

Copy



✓ Monorepo

- One cache and remote: maximum deduplication
- Cross-project dependencies are just paths
- One CI configuration to maintain
- Atomic changes across projects

✗ Separate repos + import

- Teams version and release independently
- Consumers pin an explicit upstream revision
- Access control per repository
- But: duplicated cache, and update is manual

Choose on ownership, not on tidiness

If one team owns everything, a monorepo is simpler and cheaper. If a dataset has its own owner and its own release cadence, give it its own repository and let consumers pin a revision. Retrofitting either direction is painful, so decide before there are five projects.

► TRY IT

1. Create two subdirectories each with their own `dvc.yaml`, and run `dvc repro --all-pipelines`.
2. Make a stage in the second project depend on an output from the first, and confirm `dvc dag` shows the link.
3. Check that both share one `.dvc/cache` by looking for both outputs' hashes in it.
4. Now try the other shape: `dvc import` from a second local repository with `--rev`.

Expected result – two pipelines in one repository sharing a cache, with a cross-project dependency visible in the DAG. Building both layouts once makes the trade-off concrete rather than theoretical.

Debugging, one level deeper



BASH

Copy

```
dvc repro --dry                # what would run, and in what order
dvc repro --downstream train   # train and everything after it
dvc repro --force-downstream   # rerun downstream even if hashes matched
dvc repro --single-item train  # exactly one stage, no upstream
dvc repro --pull               # fetch missing deps from the remote as needed
dvc repro --no-commit          # run but do not save outputs to the cache
dvc repro -v                   # verbose: every hash comparison

dvc data status --granular --unchanged # per-file, including what did not change
dvc diff HEAD~3                    # which tracked files changed between revisions
dvc diff --targets data/ HEAD~1
```

`dvc data status --granular` is the underused one. Where `dvc status` says "modified: data/images", the granular form lists the individual files added, modified, and deleted, which is how you discover that a "changed dataset" is one corrupted image.

Four subtle failures and how to find them:

Symptom	Real cause	How to confirm
A stage reruns every time	A dependency whose hash changes on every run: a timestamped log, a generated file	<code>dvc repro -v</code> and read which dep differed
A stage never reruns after a real change	The changed input is not in <code>deps</code>	<code>dvc dag</code> and compare against what the script reads
Results differ between machines	Undeclared randomness or an unpinned library	Seed everything and add <code>requirements.txt</code> to <code>deps</code>
<code>dvc.lock</code> conflicts on every merge	Two branches both reran the pipeline	Expected; regenerate with <code>dvc repro</code> after merging rather than hand-resolving

Never hand-resolve a `dvc.lock` merge conflict



inconsistent data. Take one side, usually theirs, then run `dvc repro` and commit the regenerated lock.

► TRY IT

1. Add a dependency that changes every run, a file containing the current timestamp, and confirm the stage never caches.
2. Find it with `dvc repro -v` and read the hash comparison.
3. Run `dvc data status --granular` on a directory where you changed one file out of many.
4. Create a `dvc.lock` conflict deliberately on two branches, then resolve it the correct way: take one side and repro.

Expected result – a stage that can never be cached, found in one verbose run, and a granular status that names the single changed file. The lock conflict exercise is worth doing before it happens for real on a Friday.

Putting it all together

Everything from this level in one project. Nothing here is new. Read it as a whole and you should be able to justify every line.

params.yaml

Copy

```
prepare:
  test_size: 0.2
  random_state: 42

featurize:
  methods: [tfidf, embeddings]
  max_features: 50000

train:
  n_estimators: 300
```



dvc.yaml

Copy

```
vars:
  - params.yaml
  - config/paths.yaml:paths

stages:
  # External, dated source – immutable, so the result stays reproducible
  ingest:
    cmd: python src/ingest.py
    deps:
      - src/ingest.py
      - s3://landing/exports/2024-05-01/events.parquet
    outs:
      - data/raw.parquet

  prepare:
    cmd: python src/prepare.py
    deps: [src/prepare.py, data/raw.parquet]
    params: [prepare.test_size, prepare.random_state]
    outs:
      - data/train.parquet
      - data/test.parquet

  # One block, two generated stages
  featurize:
    foreach: ${featurize.methods}
    do:
      cmd: python src/featurize.py --method ${item}
      deps: [src/featurize.py, data/train.parquet]
      params: [featurize.max_features]
      outs:
        - data/features-${item}.pkl:
            push: false # regenerable in minutes; not worth storing

  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - requirements.txt # pinned deps are a real dependency
      - data/features-tfidf.pkl
    params: [train.n_estimators, train.max_depth, train.learning_rate]
    outs:
```



```
metrics:
  - dvclive/metrics.json:
    cache: false          # small text → Git, diffable
plots:
  - dvclive/plots/metrics:
    x: step
    cache: false

evaluate:
  cmd: python src/evaluate.py
  deps: [src/evaluate.py, models/model.pkl, data/test.parquet]
  metrics:
    - metrics.json:
      cache: false
  plots:
    - plots/confusion.csv:
      template: confusion
      x: actual
      y: predicted
      cache: false
```

BASH

Copy

```
# .dvc/config - committed
[core]
  remote = origin
  autostage = true
['remote "origin"']
  url = s3://prod-bucket/dvc
  jobs = 16
['remote "archive"']
  url = s3://glacier-bucket/dvc
[cache]
  type = reflink,copy
  protected = true
```

BASH

Copy

```
# The everyday loop
dvc repro          # only what changed
```



```
git commit -m "Deeper trees, tfidf features: F1 0.910 → 0.931"
dvc push && git push

# A sweep, in parallel, isolated
dvc exp run --queue -S train.max_depth=4,8,16,24
dvc exp run --run-all --jobs 4
dvc exp show --only-changed --sort-by f1 --sort-order desc
dvc exp branch exp-a1b2c deeper-trees
dvc exp remove --all && dvc gc --dry-run --all-commits
```

Ten decisions in there are the whole lesson of this level:

Decision	Section
An immutable dated external source	External data and <code>import-url</code>
<code>foreach</code> over <code>\${featurize.methods}</code>	Pipeline templating
<code>push: false</code> on cheap regenerable intermediates	Output modifiers
<code>persist: true</code> on the training log	Output modifiers
<code>requirements.txt</code> declared as a dependency	Debugging, one level deeper
<code>cache: false</code> on metrics and DVCLive plots	Output modifiers · DVCLive
DVCLive for per-epoch curves	DVCLive
<code>jobs = 16</code> on the remote	Multiple remotes
<code>cache.protected = true</code>	Link types, measured
A queued, parallel, isolated sweep	Experiments at scale

► TRY IT: THE ONE THAT MATTERS

1. Rebuild a real project against this template and get `dvc repro` to report up to date on a second run.



review.

4. Put the CI workflow in place and confirm the `dvc status` gate fails when you forget to repro.

Expected result – a pipeline that reruns the minimum, a sweep that runs four-wide without corrupting itself, and CI that refuses a stale lock file. Those three properties are what separate a pipeline that works from one a team can operate.

Where you are now

You can explain the cache layout and why hashing dominates, choose link types deliberately, template a pipeline instead of copy-pasting stages, use every output modifier correctly, depend on external and versioned sources, tier storage across remotes, run parallel isolated sweeps and share them, log training dynamics with DVCLive, render declarative plots for reviewers, enforce reproducibility in CI, lay out a monorepo, and debug the subtle failures where nothing errors.

Can you...	
Say what dominates <code>dvc add</code> time?	Hashing, not transfer
Explain the <code>files/md5/xx/</code> layout and <code>.dir</code> objects?	Two-char shard, JSON listing per directory
Give the hardlink corruption failure?	Working file and cache share an inode
Collapse ten similar stages into one block?	<code>foreach</code> or <code>matrix</code>
Name four output modifiers and their use?	<code>cache: false</code> , <code>persist</code> , <code>push: false</code> , <code>remote:</code>
Say why an external dep can break reproducibility?	A moving path can be overwritten
Make <code>dvc pull</code> faster?	<code>jobs</code> , and cache <code>.dvc/cache</code> in CI



Run a sweep four-wide safely:	Queue plus <code>--temp</code> isolation
Say what DVCLive adds over <code>metrics.json</code> ?	Per-step curves, not just endpoints
Name the CI check that enforces reproducibility?	<code>dvc status</code> must be empty
Resolve a <code>dvc.lock</code> conflict correctly?	Take one side, then <code>dvc repro</code>
Find which single file in a directory changed?	<code>dvc data status --granular</code>

Senior takes every one of these further, with a security, scale, or ownership dimension.

Who can read and who can write each remote, and how credentials stop being stored at all. Immutability, retention, and object-lock on the storage side, and what that means for `dvc gc`. Cost modelling: storage class, egress, and the real price of a 200 GB intermediate. Hashing and transfer at terabyte scale, and when to stop using DVC for a given dataset. Data registries as a product with a release cadence. Lineage and audit trails for regulated work. GDPR deletion against a content-addressed, immutable cache. CI/CD with GPU and self-hosted runners. Where DVC stops and a feature store, lakehouse, or table format begins.



PREVIOUS

Beginner · Tips & Tricks

NEXT UP

Mid-level · Interview Review





MLOps MENA
COMMUNITY

العربية



Join the Community ↗



LEARN

Roadmaps

Student Guides

Courses

The MLOps Practitioner

Sessions

Articles

COMMUNITY

Meet the team

Services for companies

FAQ

Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**



MLOps MENA
COMMUNITY

العربية



Join the Community ↗

